13 | 模型提供商管理：第一个完整功能的交付闭环

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

从这一讲开始，正式进入核心功能开发。

前面 12 讲都在“准备”——认知框架、产品定义、架构设计、工程骨架、基础组件、前端 UI。我们已准备得足够扎实，现在该看看回报了。

第一个模块是模型提供商管理，让用户配置 OpenAI、Claude、Gemini、Ollama 这些 LLM 提供商，管理 API Key，查看支持的模型，监控健康状态。为什么从这个开始讲起？因为它是 Hify 整个平台的地基，Agent 要选模型、对话要调模型，所有功能都依赖它。而且复杂度刚好，有 CRUD、有外部调用、有鉴权处理、有健康检查，足够展示一个完整的交付流程，但不至于让你迷失在业务细节里。

这一讲最重要的不是 Provider 模块本身，而是它背后的标准交付流程。后面做 Agent、对话引擎、MCP 接入，每个模块都复用这个流程。

我会分四个部分展开：先想清楚、后端拆解执行、前端对接、完整验收。

先和 Claude Code 想清楚

10 讲学了咨询模式，**不确定的时候先问再做**。做一个新的业务模块，正是用这个模式的好时机。**上来就让 Claude Code 写代码是最常见的错误**，你连这个模块要考虑哪些东西都没想清楚，它写出来的代码一定有遗漏。

1. 支持哪些供应商?

比如，你知道业界有多少 LLM 的供应商吗？我敢保证你肯定不知道，或者不知道全。所以你可以问 Claude Code：

> Hify 要支持 LLM 模型提供商管理。帮我分析：主流的 LLM 供应商有哪些？它们的 API 有什么共性和差异？哪些是一期必须支持的?

Claude Code 给了一份非常详细的分析。信息量最大的不是供应商列表本身，而是它按三个维度做的分类，这直接影响了我们后面的架构决策。

按接口兼容性分：

分类	供应商	含义
OpenAI 原生	OpenAI	标准格式
OpenAI 兼容（只改 baseUrl）	DeepSeek、Moonshot、大多数国产模型	接口完全一样，换个地址就能用
需独立适配	Anthropic、Google Gemini、Ollama	消息格式或认证方式有差异

这个分类是关键洞察，“OpenAI 兼容”是一个巨大的阵营。DeepSeek、Moonshot、Azure OpenAI 这些，接口格式和 OpenAI 一模一样，只需要改 baseUrl 和 API Key 就能接入，不需要写一行适配代码。

按认证方式分：Bearer Token（OpenAI、DeepSeek）、自定义 Header（Anthropic 用 x-api-key）、URL 参数（Gemini）、无认证（Ollama）、JWT 自签名（智谱 GLM）。这个差异直接影响了我们 auth_config 用 JSON 存储的决策。

按消息格式分：OpenAI 格式（大多数）、Anthropic 独立 system 字段、Gemini 完全不同格式（contents + parts）。这个差异决定了后面适配层要怎么设计。

基于这个分析，我的判断，一期支持三种类型加一个通用兼容：

类型	覆盖供应商	理由
OpenAI	OpenAI	行业标准
Anthropic	Claude	主流大模型，需独立适配
Ollama	Ollama	本地部署必备
OpenAI 兼容	DeepSeek、Moonshot、Azure 等	用户只需填 baseUrl 和 Key，无需写代码

四个类型覆盖 90% 的使用场景。Gemini 消息格式差异最大，适配成本高，放二期。
“OpenAI 兼容”类型是最聪明的设计，不是每接入一个供应商就写一套适配代码，而是让用户自己配置 baseUrl，立刻能用。

你会发现，这个小节我写得很详细。这是想告诉你：**经过这个问题下来，你是不是一下子就了解了业界主流的 LLM 供应商，以及供应商提供的接口的形态和内容。而这就是你在 AI 领域知识的积累。所以“会问”很重要，通过问来学习了解一个你不知道的领域。**

最后接口兼容性的分类，后面适配层设计时会直接用到。13 讲先用 if-else 跑通，下一讲用设计模式重构。

2. 有没有现成的依赖库？

> Java 生态里有没有封装了多 LLM 供应商调用的库？Spring AI、LangChain4j 等，分析成熟度和优缺点。

Spring AI 和我们技术栈最匹配，但 API 还在快速迭代，LangChain4j 功能全但概念太重。

我的判断：一期不引入这些框架，基于 10 讲封装的 LlmHttpClient 自己做。大部分供应商兼容 OpenAI 格式，自己封装工作量不大。引入大框架只为用模型调用部分，性价比不高。

这个决策过程本身值得学习，不是有轮子就一定要用，要看轮子和你场景的匹配度。

3. 数据模型设计

这是最关键的部分，数据怎么存，决定了接口怎么设计，也决定了前端怎么展示。

> 基于上面的分析，设计 Provider 模块的数据模型。需要考虑：多种供应商鉴权方式的差异怎么统一存储、一个供应商下有多个模型怎么管理、供应商健康状态怎么表示。

Claude Code 给了一版很扎实的设计（太细致了，我就先不贴内容了）。总结就是有几个点，比我预期的更好。逐个说。

鉴权信息怎么存?

不同供应商的鉴权差异很大。OpenAI 用 API Key，Anthropic 额外需要 anthropicVersion Header，Azure 需要 resourceName 和 apiVersion，Ollama 完全不需要认证。给每种方式定固定列行不通。

用 `auth_config` JSON 字段，按 type 存不同结构：

复制代码

```
// OPENAI / OPENAI_COMPATIBLE
{ "apiKey": "sk-xxx" }

// ANTHROPIC
{ "apiKey": "sk-ant-xxx", "anthropicVersion": "2023-06-01" }

// OLLAMA
{}
```

未来加新供应商零改表，JSON 让每种 type 按自己的 schema 存。Claude Code 一开始建议单独建 auth 表，我追问“一对一关系建表有什么收益”，它承认没有实际收益。**这种细节层面的追问就是架构师的价值。**

模型列表怎么管理?

model_config 表有两个容易混淆的字段：`name` 是展示名（比如 GPT-4o），`model_id` 是调用时实际传给 API 的值（比如 gpt-4o）。Azure 场景下 model_id 就是 deployment name，和展示名不一样。另外加了 `context_size` 字段存上下文窗口大小，后面对话引擎做上下文管理时直接用，不需要再去查。`enabled` 字段让用户选择启用哪些模型开放给 Agent。

健康状态为什么独立成表?

这是 Claude Code 给的一个我没想到的好设计。健康状态写频繁——定时探测每分钟更新一次，每次 LLM 调用也可能更新状态。如果放在 provider 表里，高频写操作会和业务读竞争锁。分离之后，provider 表写少读多，可以放心加 @Cacheable 缓存；provider_health 表不缓存，直接读库。

而且 provider_health 表的字段比简单的 status 丰富得多：`fail_count`（连续失败次数，配合熔断器）、`latency_ms`（最近延迟）、`last_success_at`（最后成功时间）、`error_message`（最近失败原因）。这些信息在管理控制台展示时非常有用。

最终表结构：

复制代码

```
-- 模型提供商
CREATE TABLE provider (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL COMMENT '供应商名称，唯一',
    type VARCHAR(30) NOT NULL COMMENT 'OPENAI/ANTHROPIC/OLLAMA/OPENAI_COMPATIBLE'
    base_url VARCHAR(500) NOT NULL COMMENT 'API 基础地址',
    auth_config JSON COMMENT '鉴权配置，结构按 type 不同',
    enabled TINYINT DEFAULT 1 COMMENT '0=禁用 1=启用',
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,
```

```sql
    deleted TINYINT DEFAULT 0
) COMMENT '模型提供商';

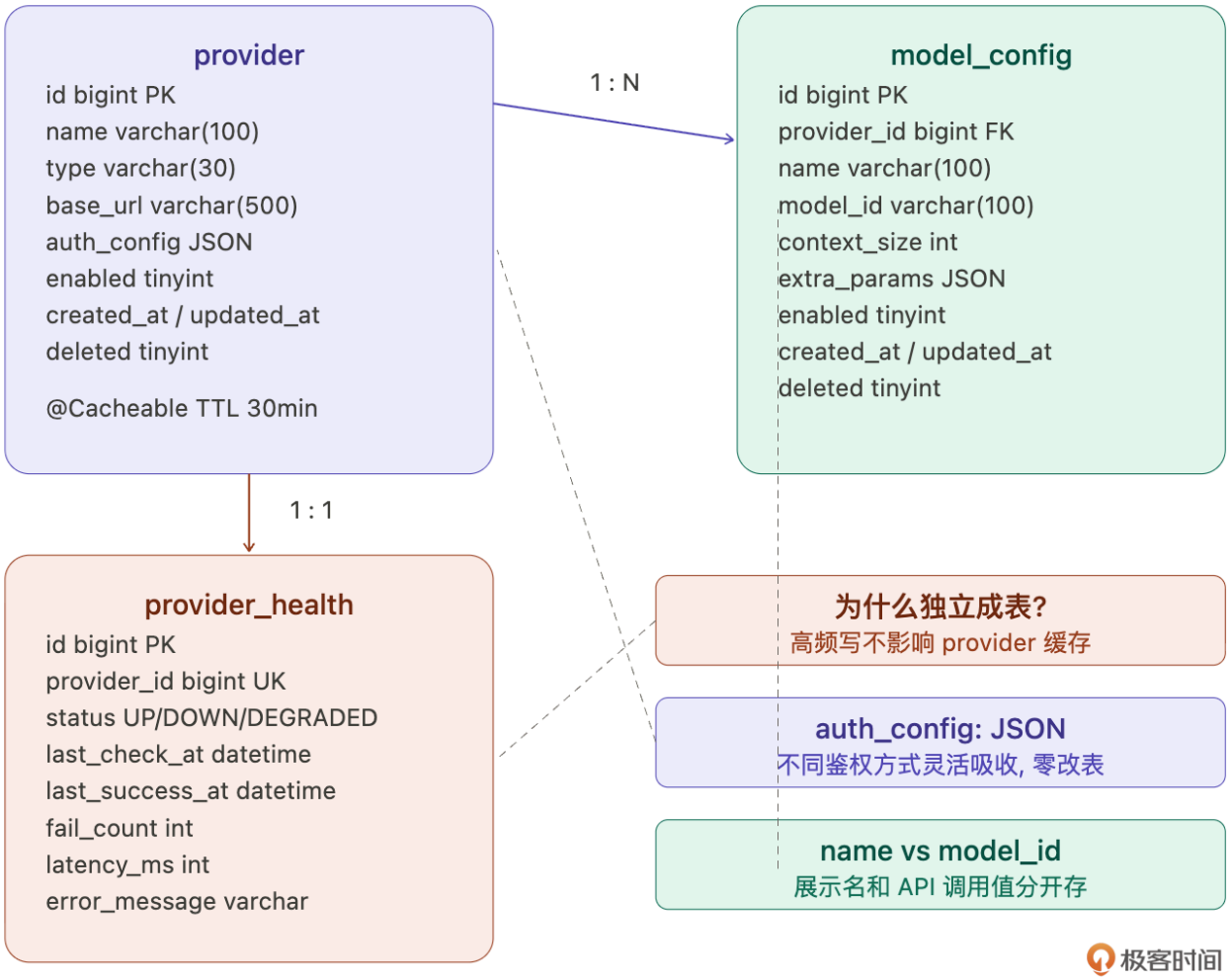
-- 模型配置
CREATE TABLE model_config (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    provider_id BIGINT NOT NULL COMMENT '所属供应商 ID',
    name VARCHAR(100) NOT NULL COMMENT '展示名，如 GPT-4o',
    model_id VARCHAR(100) NOT NULL COMMENT '调用时传给 API 的值',
    context_size INT COMMENT '上下文窗口大小（token 数）',
    extra_params JSON COMMENT '模型级别扩展参数',
    enabled TINYINT DEFAULT 1 COMMENT '0=禁用 1=启用',
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,
    deleted TINYINT DEFAULT 0
) COMMENT '模型配置';

-- 供应商健康状态（独立表，高频写不影响 provider 缓存）
CREATE TABLE provider_health (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    provider_id BIGINT NOT NULL COMMENT '供应商 ID，唯一索引',
    status VARCHAR(20) DEFAULT 'UNKNOWN' COMMENT 'UP/DOWN/DEGRADED/UNKNOWN',
    last_check_at DATETIME COMMENT '最后探测时间',
    last_success_at DATETIME COMMENT '最后成功时间',
    fail_count INT DEFAULT 0 COMMENT '连续失败次数',
    latency_ms INT COMMENT '最近一次延迟 ms',
    error_message VARCHAR(500) COMMENT '最近失败原因',
    updated_at DATETIME NOT NULL,
    UNIQUE INDEX idx_provider_health_provider_id (provider_id)
) COMMENT '供应商健康状态';
```

实体关系：

复制代码

```
provider (1)
  ├── auth_config: JSON          鉴权，结构随 type 变化
  ├── model_config (N)           该供应商下的所有模型
  │     └── extra_params: JSON   模型级别扩展
  └── provider_health (1)        健康状态，高频更新独立
```



后端拆解执行

想清楚了，开始做。08 讲的任务拆解方法论直接复用——按 MVC 分层拆，每个任务只涉及一层，产出物可独立验证，从底层往上层搭。

顺序	任务	产出	验证方式
1	Entity + Mapper	Provider、ModelConfig、ProviderHealth 实体和 Mapper	编译通过
2	DTO	请求 / 响应对象	编译通过
3	Service — CRUD	Provider 增删改查	curl 测试
4	Service — 连通性测试	调供应商 Models 接口	手动测试真实供应商
5	Service — 模型列表同步	连通后拉取模型写入 model_config	检查数据库
6	Service — 健康检查	定时任务探测状态，更新 provider_health	查看状态更新
7	Controller	RESTful 接口	curl 全接口测试
8	关联校验	删除时检查 Agent 引用	手动测试

8 个任务，两三个小时全部交付。这就是基础设施搭好之后的速度——**10 讲的 BaseEntity、PageHelper、Spring Cache、入参校验全部直接复用**，每个任务只需要写业务逻辑，不需要重复处理基础设施。

我挑几个有代表性的任务展示。

任务 1：Entity + Mapper

> 按照 CLAUDE.md 规范和上面的表结构，在 hify-provider 中创建 Provider、ModelConfig、ProviderHealth 的 Entity 和 Mapper。Entity 继承 BaseEntity（ProviderHealth 除外，它有自己的字段结构），Mapper 继承 BaseMapper。auth_config 和 extra_params 字段用 MyBatis-Plus 的 JSON TypeHandler。

这个任务很简单，Claude Code 的输出基本不用改。review 重点：字段类型和表结构一致，包路径正确。

任务 3：Service — CRUD

> 在 hify-provider 中实现 ProviderService。CRUD 基础逻辑：创建时校验名称不重复，列表支持按 type 和 enabled 筛选，详情接口返回关联的 modelConfig 列表和

> providerHealth 信息。缓存：列表和详情加 @Cacheable(cacheNames = “provider-cache”)，更新和删除加 @CacheEvict。

注意缓存。10 讲配好了 Spring Cache，这里直接加注解就行。不需要手写 if-else 查缓存逻辑。provider_health 不走缓存，直接读库。

任务 4：连通性测试

这个稍微复杂，不同供应商的 API 不一样。

> 在 hify-provider 中实现连通性测试。根据 provider.type 分发到不同的测试方法：openai 和 openai_compatible 调 GET /v1/models（Bearer Token），anthropic 调 GET /v1/models（Header 带 x-api-key + anthropic-version），ollama 调 GET /api/tags（无认证）。统一返回 ConnectionTestResult（success、latencyMs、modelCount、errorMessage）。使用 10 讲封装的 LlmHttpClient，超时 10 秒。

下面是 Claude 的产出，主要就是完成了 Client 的封装。

编译通过。产出汇总：

文件	说明
LlmHttpClient	新增 `get(url, headers, client)` 方法，client 由调用方传入以支持自定义超时
ConnectionTestResult	结果 DTO，静态工厂 `ok()` / `fail()`
ProviderConnectionTestService	分发逻辑 + 各供应商实现

各供应商测试细节：

类型	端点	认证方式	解析字段
OPENAI / OPENAI_COMPATIBLE	`{baseUrl}/v1/models`	`Authorization: Bearer {apiKey}`	`data[]` 数组长度
AZURE_OPENAI	`{baseUrl}/openai/models?api-version={apiVersion}`	`api-key: {apiKey}`	`data[]` 数组长度
ANTHROPIC	`{baseUrl}/v1/models`	`x-api-key` + `anthropic-version`	`data[]` 数组长度
OLLAMA	`{baseUrl}/api/tags`	无	`models[]` 数组长度

`testClient` 独立于 `LlmHttpClient` 内部的 `httpClient`，connect 5s + read **10s**，只用于连通性测试，不影响正常 LLM 调用。

代码很多我们就不一一展开讲了，有兴趣去看我们的源码目录。下面是测试选择的代码实现：展示连通性测试 Service 的核心逻辑——根据 type 分发到不同测试方法。

```java
public ConnectionTestResult test(Provider provider) {
    long start = System.currentTimeMillis();
    try {
        return switch (provider.getType().toUpperCase()) {
            case "OPENAI", "OPENAI_COMPATIBLE", "DEEPSEEK" -> testOpenAiCompatible(provider, start);    "DEEPSEEK": Unknown word.
            case "AZURE_OPENAI"                            -> testAzure(provider, start);
            case "ANTHROPIC"                               -> testAnthropic(provider, start);
            case "OLLAMA"                                  -> testOllama(provider, start);    "OLLAMA": Unknown word.
            default -> ConnectionTestResult.fail("不支持的供应商类型: " + provider.getType());
        };
    } catch (LlmApiException e) {
        log.warn("连通性测试失败 provider={} type={}: {}", provider.getName(), e.getType(), e.getMessage());
        return ConnectionTestResult.fail(e.getMessage());
    } catch (Exception e) {
        log.warn("连通性测试异常 provider={}: {}", provider.getName(), e.getMessage());
        return ConnectionTestResult.fail("测试异常: " + e.getMessage());
    }
}
```

这里不同供应商的差异用 if-else 处理。先跑通，下一讲我们专门讲怎么用设计模式重构它。

任务 6：健康检查定时任务

> 在 hify-provider 中实现供应商健康检查定时任务。@Scheduled 每分钟执行一次，遍历所有 enabled 的 provider，调连通性测试方法。成功则更新 provider_health：status=UP、latency_ms、last_success_at、fail_count 归零。失败则 fail_count+1，连续失败 3 次标记 DOWN。使用 asyncExecutor 线程池异步执行，不阻塞主线程。

输出的核心逻辑是：

编译通过。几个设计说明：

- **`fixedDelay` vs `fixedRate`**：用 `fixedDelay = 60_000`，上次执行完成后再等 60s，避免前一轮还没跑完下一轮就开始（供应商多时可能出现）
- **`@Qualifier` + `@RequiredArgsConstructor`**：Lombok 不支持在字段上同时用两者注入，这里用了字段注入 + `@Qualifier`，实际注入由 Spring 处理。如果项目 lint 要求构造函数注入，可以改为手动写构造函数
- **状态转换**：`UNKNOWN → DEGRADED（1–2次失败）→ DOWN（≥3次）`；成功立即回 `UP`，`fail_count` 归零

任务 7：Controller

> 在 hify-provider 中创建 ProviderController，按照 CLAUDE.md 接口规范实现所有接口：POST 创建、GET 列表（分页）、GET 详情（含 modelConfig 和 health）、PUT 更新、DELETE 删除、POST /{id}/test-connection 连通性测试。所有接口返回 Result<T>，入参加 @Valid 校验。

输出是：

编译通过。最终接口列表：

方法	路径	说明
POST	/api/v1/providers	创建，@Valid 校验 name/type/baseUrl/authConfig
GET	/api/v1/providers?type=&enabled=&page=&pageSize=	分页列表
GET	/api/v1/providers/{id}	详情，含 models 和 health
PUT	/api/v1/providers/{id}	更新，@Valid 校验
DELETE	/api/v1/providers/{id}	删除
POST	/api/v1/providers/{id}/test-connection	连通性测试

因为我们 10 讲跑通的 DemoItem 标准流程在这里完全复用——Controller 只做参数校验和调 Service，返回 Result，分页用 PageHelper。DemoItem 就是 Provider 的模板，模式一模一样，只是字段和业务规则不同。

后端 8 个任务全部完成后，用 curl 快速过一遍：

复制代码

```
# 创建 OpenAI 供应商
curl -X POST http://localhost:8080/api/v1/providers \
  -H "Content-Type: application/json" \
  -d '{"name":"OpenAI","type":"OPENAI","baseUrl":"https://api.openai.com","authCo

# 连通性测试
curl -X POST http://localhost:8080/api/v1/providers/1/test-connection

# 查看详情（包含模型列表和健康状态）
curl http://localhost:8080/api/v1/providers/1

# 分页列表
curl "http://localhost:8080/api/v1/providers?page=1&pageSize=10"
```

在运行这几个 curl 命令的过程中，发生了很多个错误。



四个接口全部通过。汇总一下这次修复过程中解决的问题：

问题	根因	修复
Spring Boot 启动无日志输出	`logback-spring.xml` 无 mock profile 的 appender	加入 `mock` 到彩色控制台 profile
`%clr` 转换器报错	直接用 `java -jar` 时 Spring Boot 扩展未注册	改为普通 `%d` pattern
`cacheManager` Bean 冲突	`RedisConfig` 的 `@ConditionalOnProperty` 条件在 mock 下仍满足	改为 `@Profile("!mock")`
`factoryBeanObjectType` 错误	`mybatis-plus-boot-starter` 与 Spring Boot 3.2 不兼容	换用 `mybatis-plus-spring-boot3-starter` + `mybatis-plus-jsqlparser`
`PaginationInnerInterceptor` 找不到	3.5.9 把分页拦截器移到 `jsqlparser` 模块	加 `mybatis-plus-jsqlparser` 依赖
`RedisUtil` Bean 找不到	mock 排除了 Redis 自动配置	加 `@Profile("!mock")`
`@PathVariable` 参数名无法识别	编译缺少 `-parameters` flag	父 pom compiler plugin 加 `<parameters>true</parameters>`
端口占用	旧进程没停	`lsof -ti:8080

这个后台任务已经完成（依赖下载成功），我们已经在之后的步骤里用上了它。不需要再读取输出文件。

这个后台任务是中间过程的失败记录，已经在后续步骤中解决了，不需要处理。

不过这些问题我都是让 Claude Code 自己修复。最后后端跑通了。到这里我真的想感叹一句：**时代真的变了，以前我们解决这些问题至少要一两天，现在 AI 半个小时就搞定了**。挺可怕的，我们研发要怎么应对这个时代呢？我也在探索，后面给我的答案，先一起努力吧。

但这不是交付闭环，用户不是在终端里用 curl 的，他们在浏览器里操作。

前端对接：把 mock 换成真实 API

11 讲我们已经做了一个交互完整的 ProviderList 页面——HifyTable 展示列表、HifyFormDialog 弹新增 / 编辑表单、useConfirm 做删除确认。数据是 mock 的，现在换成真实 API。

这一步非常快，因为 11 讲的前端基础组件已经把脏活全封装了。

> 把 ProviderList.vue 的 mock 数据换成真实 API 调用。具体改动：
>
> 1. 在 src/api/provider.ts 中创建 API 方法：getProviderList（分页）、createProvider、updateProvider、deleteProvider、testConnection
> 2. HifyTable 的 api prop 从 mock 函数换成 getProviderList
> 3. HifyFormDialog 的 submit 事件处理从 console.log 换成 createProvider/updateProvider
> 4. 删除按钮的 useConfirm 从 mock 换成 deleteProvider

5. 列表加一列“操作”：连通性测试按钮，点击调 testConnection，结果用 ElMessage 提示
6. 加一列“健康状态”：从 provider_health 关联查询，UP 绿色 tag、DOWN 红色 tag、DEGRADED 黄色 tag、UNKNOWN 灰色 tag，显示最近延迟 ms
7. 加一列“模型数”：显示该供应商下已启用的模型数量，点击可展开模型列表

1 条指令，7 个改动点。Claude Code 生成后，前端代码的改动量其实很小，API 文件是新增的，ProviderList.vue 基本只是把数据源从 mock 函数换成了 API 函数，组件结构不变。

这就是 11 讲封装前端基础组件的回报。HifyTable 已经处理了分页、loading、空状态，useRequest 已经处理了 loading/error 三态，useConfirm 已经处理了确认弹窗 + 调接口 + 成功提示。业务页面只需要关心“调哪个 API、显示哪些字段”，不需要重复处理交互逻辑。

完整验收

启动后端和前端，打开浏览器 `http://localhost:5173`：

1. 点“模型管理”，看到空列表（还没有数据）。
2. 点“新增提供商”，弹出表单，填入 OpenAI 的信息（名称、类型选 OpenAI、baseUrl、API Key），提交。
3. 列表刷新，出现一条记录，健康状态显示灰色“UNKNOWN”。
4. 点“连通性测试”，几秒后返回结果：如果 API Key 有效，提示成功并显示延迟和模型数量；健康状态变成绿色“UP”，延迟列显示具体毫秒数。
5. 点模型数展开，看到从 OpenAI 拉取的模型列表（gpt-4o、gpt-3.5-turbo 等）。
6. 等一分钟，检查健康状态是否被定时任务自动更新（看 provider_health 表的 last_check_at）。
7. 再添加一个 Ollama 供应商（类型选 Ollama，baseUrl 填 http://localhost:11434），测试连通。
8. 再添加一个 DeepSeek（类型选 OpenAI 兼容，baseUrl 填 https://api.deepseek.com），验证“OpenAI 兼容”类型开箱即用。

9. 分页、编辑、删除的常规操作验证。



从表结构设计到接口实现到前端展示，一条数据从数据库流转到浏览器，这才是完整的交付闭环。

回头看一下速度：想清楚大概一个小时（供应商分析、数据模型设计），后端 8 个任务一个小时，前端对接半小时。一个完整的业务模块，从零到前后端全部跑通，半天时间。 这就是基础设施搭好之后的开发节奏——基础组件全部复用，CLAUDE.md 规范保证输出质量，你的精力全部集中在业务决策上。

后面做 Agent、对话引擎、MCP，每个模块都是这个节奏。

总结

这一讲做了两件事：**和 Claude Code 一起想清楚 Provider 模块的设计决策，然后按标准流程从后端到前端完整交付。**

前半段“想清楚”才是最有价值的部分。一期支持三种类型加一个通用兼容、不引入大框架自己封装、鉴权信息用 JSON 灵活存储、模型列表分展示名和调用 ID、健康状态独立成表避免锁竞争，每个决策都有分析过程和判断依据。

后半段“执行”体现了基础设施的回报。08–11 讲搭的所有基础设施在这里全部兑现：BaseEntity 省了公共字段、PageHelper 省了分页逻辑、@Cacheable 一行注解搞定缓存、LlmHttpClient 直接调外部 API、HifyTable 和 HifyFormDialog 让前端对接只改数据源。半天时间，前后端完整交付。

标准交付流程：咨询模式想清楚（供应商选型、数据模型、设计决策）→ 按层拆解（Entity → DTO → Service → Controller）→ 逐步执行验证 → 前端对接 → 完整验收。后面每个模块都是这个流程，区别只是业务复杂度不同。

你可能注意到，连通性测试里不同供应商的 API 差异用了 if-else 处理。这是刻意的，先跑通再优化。下一讲我们回过头来，看看 13 讲积累了哪些可复用的流程，把它们沉淀下来，同时讲清楚怎么用设计模式重构那些 if-else。

思考题

这一讲的 Provider 模块有几个刻意没展开的边界问题，试着用 Claude Code 思考：

1. 如果用户修改了 API Key，@Cacheable 的缓存还没过期，对话服务用的还是旧 Key，这个缓存一致性问题怎么处理？
2. 健康检查发现供应商 DOWN，但有正在进行的对话还在用它，应该中断还是让它继续？
3. API Key 明文存在数据库里有安全风险，生产环境怎么做加密存储？

可以把你和 Claude Code 讨论的过程分享在留言区。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (6)

最新　精选



s(t)
2026-07-05 来自上海

老师我现在遇到了个比较棘手的问题，就是我卡在这章蛮久的了。我自己去咨询大模型的话，可能更多的是问它应该怎么做，而不是像您这样带有目的性、指向性的让他怎么做。然后大模型就会吐出超级多的内容给我，然后我作为一名前端，就没法判断他给我的内容哪些是需要的，哪些是可以不管的。我也试过让他给我优先级最高的几个，但还是有很多内容都是你文章

里面没有提及的，我应该怎么办？还是先不去判断这些，要么按照您的提示词来，先跑通整个流程。或者我直接按照他给的优先级最高的流程来先实现功能？不管他的内容和你一不一样。我理解大模型输出的内容应该都挺有道理的，可以直接参考？

作者回复: 你这个问题其实就是：我们读书时跟老师学一个我们全新的东西一样。
在我看来，就是：
1. 跟着老师依样画葫芦全部跑一遍，体验全流程，能学到多少学到多少
2. 然后有个AI 老师在旁边指导你/教你，跟你讲原理，为什么这么做。

对你来说：
1. 当你依样画葫芦做了一遍后，因为你有基础了（你是前端），所以你一定能建立你的理解和框架
2. 如果你在过程中对不了解的去学习、去问、去记录，那么你也一定有更深的理解
3. 然后你在过程中，你不需要纠结第二步AI老师说的对不对，你就听着，按你当前的能力理解吸收即可。

然后你做完一遍，思考完一遍，你就能学到比如50%。那你体验过一遍，再回过头重头做一遍，那么体验会更深。

我理解就是经验积累的过程。也就是这个课程给你的项目经验。

然后当你做其他项目的时候，这个课程给你的项目就能用起来了。

4



Popeye
2026-04-25 来自广东

有考虑讲怎么维护旧项目的吗？ 毕竟维护才更贴切现在企业内部的真实情况。

作者回复: 有的，在准备讲这个了，准备中了。敬请期待～

共 2 条评论>

1



庄周梦蝶
2026-06-26 来自浙江

老师，请教：自动化测试和codereview也可以让ai做吧，希望老师能加餐这块

作者回复: ok，没问题，我来安排。
是的，自动化测试和codeReivew都是让AI做的。这个项目有一些实践：https://robustmq.com/zh/

Blogs/111
有兴趣也可以看看。

Geek_91c6e4
2026-05-21 来自重庆

根据我的分析，我现在可以给你一个完整的答案了。让我总结一下 model_config 表的数据写入方式：
分析结果
经过详细查看 hify-provider 模块的代码，我发现 model_config 表的数据目前没有通过 API 接口进行写入

作者回复: 是的，因为篇幅原因，只完成了主流程，这个配置好像是没做。你可以补充下这个功能，就当练练手。



null
2026-04-17 来自北京

这个源代码在哪里看呀

 3



Y
2026-04-24 来自广东

1.如果用户修改了 API Key，@Cacheable 的缓存还没过期，对话服务用的还是旧 Key，这个缓存一致性问题怎么处理?
-- 用@CacheEvict注解删除缓存。对话服务会重新从数据库获取最新的key
2.健康检查发现供应商 DOWN，但有正在进行的对话还在用它，应该中断还是让它继续?
-- 健康检查发现供应商DOWN。正在进行的对话应该继续，系统异常了给异常提示就行。
3.API Key 明文存在数据库里有安全风险，生产环境怎么做加密存储?
-- 加密密钥放在配置中心上，数据库只存储加密后的API KEY

 1